

Azerbaijani Bank Logo Detection

Project Documentation

Avaz Asgarov

January 15, 2026

Table of Contents

1	Introduction	3
1.1	Project Description	3
1.2	Project Objectives	3
1.3	Scope and Limitations	3
2	Theoretical Background	3
2.1	Object Detection Overview	3
2.2	Architectures Used	4
2.2.1	YOLO (You Only Look Once)	4
2.2.2	Faster R-CNN	5
3	Methodology	5
3.1	System Architecture	5
3.2	Data Acquisition	6
3.2.1	Data Sources	6
3.2.2	Dataset Distribution	6
3.3	Synthetic Data & Limitations	7
3.3.1	Addressing Data Scarcity	7
3.3.2	Synthetic Generation Techniques	7
3.3.3	Limitations and Trade-offs	8
3.4	Data Annotation	8
3.4.1	Annotation Tool and Protocol	8
3.4.2	Annotation Formats	8
4	Experimental Setup	9
4.1	Models Configured	9
4.1.1	YOLOv5 (One-Stage, Anchor-Based)	9
4.1.2	YOLOv8 (One-Stage, Anchor-Free)	10
4.1.3	Faster R-CNN (Two-Stage)	10
4.2	Training Configuration	10
4.2.1	Hyperparameters	10
4.2.2	Data Augmentation	11
4.2.3	The YAML Configuration	11
5	Results and Discussion	12
5.1	Evaluation Metrics	12
5.2	Quantitative Analysis	12
5.3	Confusion Matrices	13
5.3.1	Analysis of Model Behavior	14
5.4	Qualitative Analysis	14
5.4.1	Successful Detections	14
5.4.2	Failure Cases and False Positives	14
6	Deployment: The AI Dashboard	16
6.1	Streamlit Interface	16
6.1.1	Control Panel (Sidebar)	16
6.1.2	Real-Time Inference Display	17
6.2	Intelligent Forensic Analysis (OpenAI Integration)	17
6.2.1	The Innovation: Visual Scene Understanding	17
6.2.2	Prompt Engineering Strategy	17

6.2.3	Forensic Output Example	18
7	Conclusion and Future Work	19
7.1	Conclusion	19
7.2	Future Work	19
7.2.1	Dataset Expansion (New Classes)	19
7.2.2	Real-Time Video Forensics	19
8	References	20

1 Introduction

1.1 Project Description

This project focuses on the development of an object detection system designed to identify and classify the logos of three major Azerbaijani financial institutions: the International Bank of Azerbaijan (ABB), Pasha Bank, and Kapital Bank.

The core work involved collecting a custom dataset of bank logos and training state-of-the-art deep learning models to perform multi-class detection. Specifically, the project utilizes and compares two distinct architectures: **YOLO (You Only Look Once)** and **Faster R-CNN**. The system is deployed via a Streamlit dashboard, which integrates OpenAI's GPT-4o to provide additional contextual analysis of the detected regions.



Figure 1: Target classes for the detection system: ABB, Pasha Bank, and Kapital Bank logos.

1.2 Project Objectives

The primary goals of this project are:

- **Data Collection and Training:** To compile a labeled dataset of bank logos and train object detection models to accurately localize ABB, Kapital Bank, and Pasha Bank logos.
- **Model Comparison:** To evaluate the performance of one-stage detectors (YOLOv5, YOLOv8) against two-stage detectors (Faster R-CNN) in terms of accuracy and speed.
- **Dashboard Development:** To build an interactive web interface using Streamlit that allows users to upload images and view detection results, augmented by AI-driven context analysis.

1.3 Scope and Limitations

The project operates under the following constraints:

- **Target Banks:** The detection scope is strictly limited to three specific classes: ABB, Pasha Bank, and Kapital Bank. Logos from other institutions are not recognized.
- **Dataset Size and Composition:** The training dataset is relatively small due to the limited availability of high-quality public images for these specific local banks.

2 Theoretical Background

2.1 Object Detection Overview

Object detection is a core computer vision task that builds upon simpler concepts. As illustrated in Figure 2, there are three distinct levels of understanding an image:

- **Classification:** The model simply answers "*What is in this image?*" It assigns a single label (e.g., "Cougar") to the whole picture but does not know where the object is.
- **Classification + Localization:** The model identifies the object and draws a single bounding box around it. This works well when there is only one main object.
- **Object Detection:** This is the most advanced task. The model finds *multiple* objects of different classes (e.g., a "Cougar" and a "Butterfly") at the same time and draws accurate bounding boxes around each one.

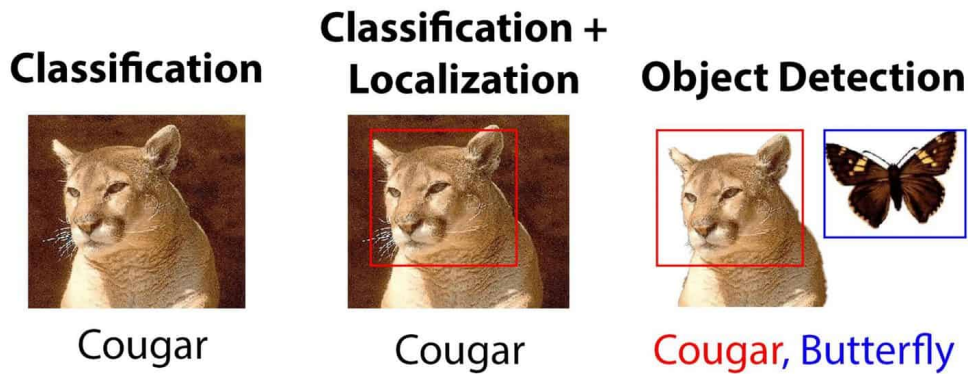


Figure 2: Visual progression from Classification to Object Detection.

2.2 Architectures Used

To achieve accurate logo detection, this project employs two different types of deep learning models: One-Stage (YOLO) and Two-Stage (Faster R-CNN).

2.2.1 YOLO (You Only Look Once)

YOLO is a **One-Stage Detector**. Imagine a human glancing at an image once and immediately pointing at every object they see. That is how YOLO works—it looks at the entire image and predicts all bounding boxes and classes in a single step. This makes it extremely fast.

Anchor vs. Anchor-Free:

- **YOLOv5 (Anchor-Based):** This version uses "Anchors," which are like pre-made template boxes of different sizes (small, medium, large). The model tries to fit objects into these templates. It is reliable but requires careful tuning of the template sizes.
- **YOLOv8 (Anchor-Free):** This newer version removes the templates. Instead of trying to fit an object into a box, it simply predicts the *center point* of an object and measures the distance to its edges. This is simpler, more flexible, and often better at detecting irregularly shaped logos.

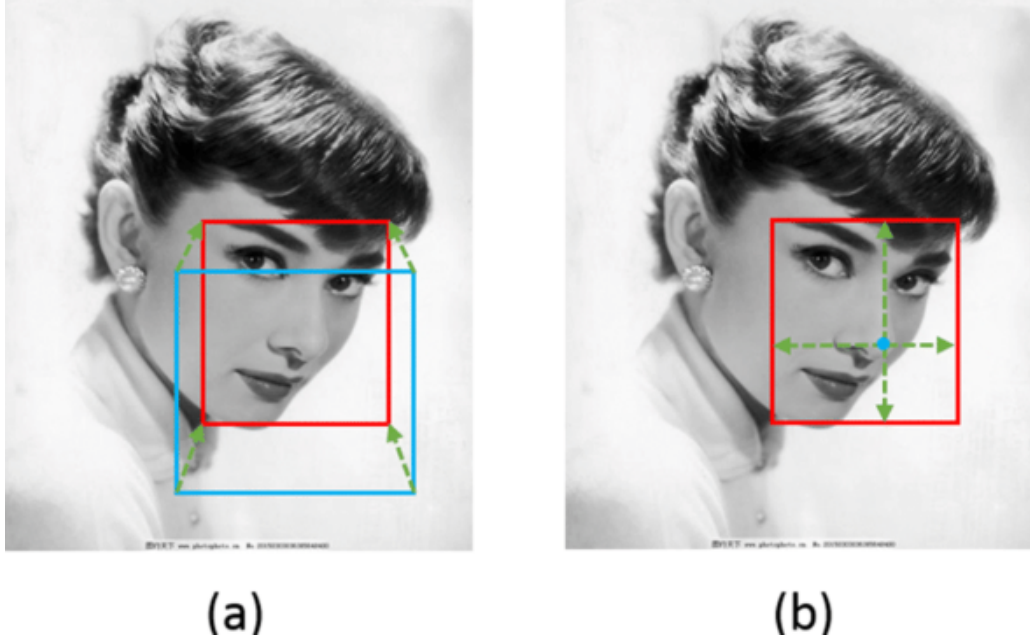


Figure 3: A visual explanation showing the difference between anchor-based and anchor-free methods. The **red bounding box** is the ground truth, the **blue bounding box** is a predefined anchor, and the **green lines** are the offsets. (a) Anchor-based methods predict offsets based on the predefined anchor. (b) Anchor-free methods directly estimate the offsets of a point to its outside boundaries.

2.2.2 Faster R-CNN

Faster R-CNN is a **Two-Stage Detector**. It works more like a careful editor:

1. **Stage 1 (Proposal)**: First, it scans the image to find "Regions of Interest"—areas that look like they might contain *something* (it doesn't care what yet).
2. **Stage 2 (Refinement)**: Then, it looks closely at each of those specific regions to classify exactly what is inside (e.g., "ABB Logo") and tightens the box around it.

Because it double-checks everything, Faster R-CNN is usually more accurate, especially for small objects, but it is significantly slower than YOLO.

Table 1: Comparison of Object Detection Architectures

Feature	YOLOv5	YOLOv8	Faster R-CNN
Detection Stage	One-stage	One-stage	Two-stage
Anchor Strategy	Anchor-based	Anchor-free	Anchor-based (RPN)
Inference Speed	Very fast	Very fast	Slow
Detection Strength	Real-time detection	Small / irregular objects	High-precision detection
Typical Use Case	Video streams	General-purpose detection	Accuracy-critical tasks

3 Methodology

3.1 System Architecture

The proposed solution implements a complete end-to-end machine learning pipeline, designed to transition raw visual data into actionable forensic insights. As illustrated in Figure 4, the

architecture is structured into five distinct phases:

1. **Data Acquisition:** The process begins with the aggregation of raw images containing logos of ABB, Kapital Bank, and Pasha Bank from various digital and physical sources.
2. **Annotation:** collected data is manually labeled using standard annotation tools to create ground truth bounding boxes in both YOLO text format and Pascal VOC XML format.
3. **Model Training:** The training phase utilizes Google Colab’s cloud GPU infrastructure to train three separate architectures (YOLOv5, YOLOv8, and Faster R-CNN) on the custom dataset.
4. **Evaluation:** Post-training models are rigorously tested against a hold-out test set to compute metrics such as Mean Average Precision (mAP) and F1-score.
5. **Deployment:** The best-performing models are integrated into a Streamlit web application. This interface features a dual-inference engine that not only detects logos but also pipelines the visual results to an OpenAI GPT-4o module for semantic context analysis.

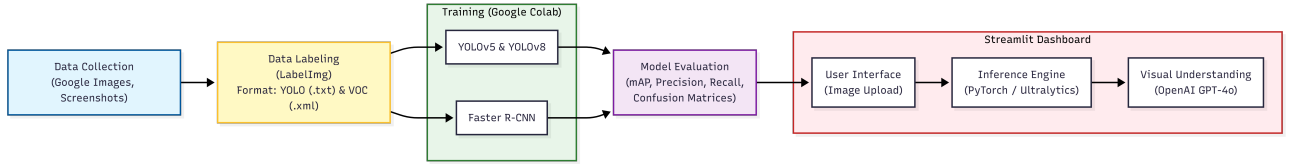


Figure 4: High-level system architecture illustrating the pipeline from data collection to AI-driven forensic analysis.

3.2 Data Acquisition

The efficacy of an object detection model is heavily dependent on the diversity and quality of its training data. For this project, a custom dataset was curated to represent the three target classes: **ABB**, **Kapital Bank**, and **Pasha Bank**.

3.2.1 Data Sources

To ensure the model generalizes well to real-world scenarios, images were aggregated from multiple diverse sources rather than relying solely on clean, digital logos:

- **Google Images:** Used to collect high-resolution variations of official branding and marketing materials.
- **Social Media Screenshots:** Real-world images taken from official Facebook and Instagram pages of the banks, capturing logos in natural environments (e.g., on merchandise, event banners).
- **Digital News Archives:** Images from news articles were included to introduce noise, such as text overlays, compression artifacts, and varying lighting conditions.

3.2.2 Dataset Distribution

The collected dataset was partitioned into three disjoint subsets to prevent data leakage and ensure rigorous evaluation. The split followed a standard ratio of approximately **70% Training**, **20% Validation**, and **10% Testing**. The final distribution is detailed in Table 2.

Table 2: Dataset Distribution by Class and Split

Class	Training (70%)	Validation (20%)	Testing (10%)
ABB	50	15	10
Kapital Bank	54	14	11
Pasha Bank	55	12	7
<i>Background / Negative</i>	<i>42</i>	<i>16</i>	<i>2</i>
Total Images	201	57	30

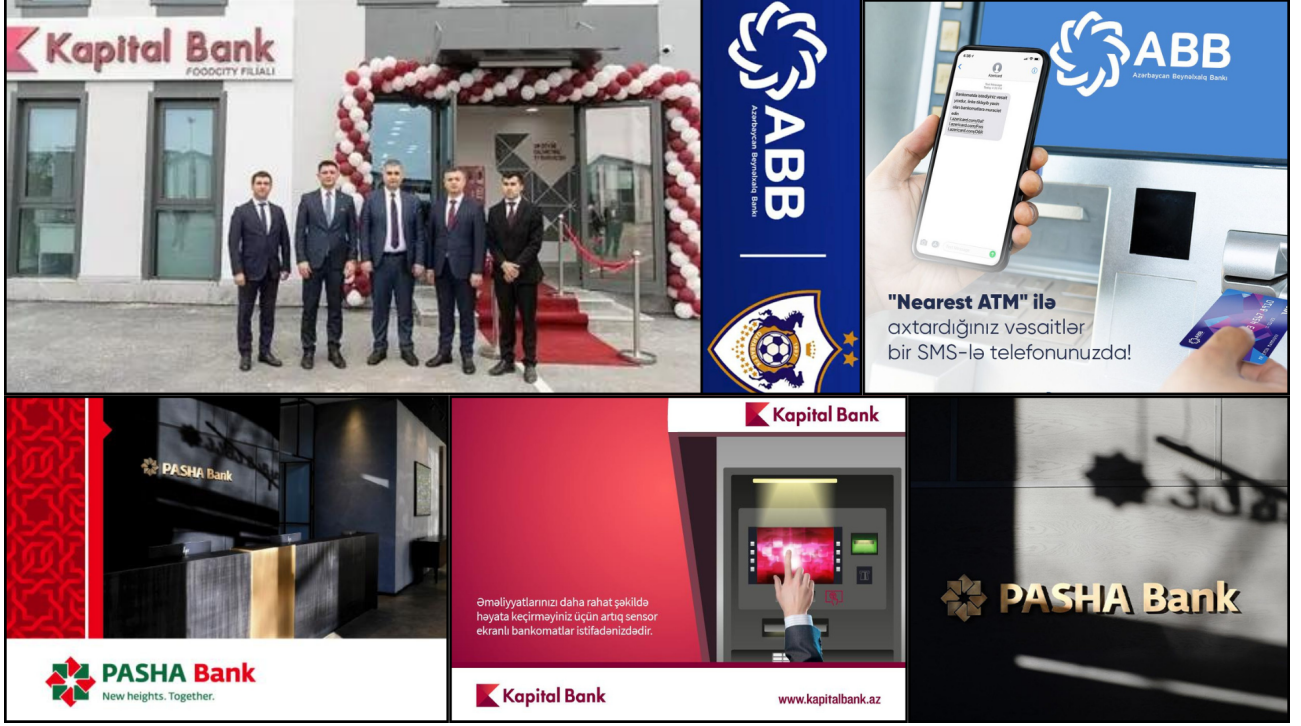


Figure 5: Examples of raw training data collected for the target classes, exhibiting variations in scale, lighting, and background clutter.

3.3 Synthetic Data & Limitations

Obtaining large-scale, diverse datasets for proprietary branding is a significant bottleneck in deep learning. This section outlines the theoretical role of synthetic data in addressing this scarcity.

3.3.1 Addressing Data Scarcity

Object detection models typically require thousands of examples to generalize effectively. For specific classes like high-quality public images are scarce and often lack real-world diversity (e.g., varying lighting, occlusions). Synthetic data generation offers a scalable solution to algorithmically enrich the training set without the costs of manual collection.

3.3.2 Synthetic Generation Techniques

The primary method for logo detection is the "Copy-Paste" Augmentation strategy.

- **Process:** A segmented mask of the target logo (e.g., a transparent PNG) is randomly pasted onto unrelated background images.

- **Transformations:** Geometric transformations such as rotation, scaling, and perspective warping are applied to simulate varied viewing angles.
- **Domain Randomization:** Lighting variations and artificial noise are introduced to improve model robustness against environmental changes.

3.3.3 Limitations and Trade-offs

While synthetic data improves recall, it introduces a **Domain Gap** between training and inference.

- **Realism:** Synthetic images often lack natural artifacts like motion blur, camera focus issues, and complex shadows.
- **Overfitting:** Models may learn to detect artificial "cut-out" edges rather than the logo features themselves if blending is imperfect.
- **Conclusion:** Consequently, while synthetic data is a valuable tool for expansion, this project prioritizes real-world data to ensure high fidelity in forensic analysis.

3.4 Data Annotation

To transform the raw image collection into a supervised learning dataset, manual annotation was performed. This process involves defining the precise spatial location and class identity of each logo within an image.

3.4.1 Annotation Tool and Protocol

The dataset was annotated using **LabelImg**, a graphical image annotation tool. The labeling process adhered to strict protocols to ensure high-quality ground truth data:

- **Tight Bounding Boxes:** Annotations were drawn to strictly enclose the visible logo pixels, minimizing the inclusion of background noise.
- **Occlusions:** Partially visible logos were labeled only if more than 50% of the brand features were distinguishable.
- **Background Images:** To reduce False Positives, the dataset includes negative samples (images with no logos). These are represented by empty annotation files, teaching the model to recognize "background" classes explicitly.

3.4.2 Annotation Formats

Since this project compares two distinct architectures, the annotations were exported in two different formats, each serving a specific model requirement:

1. YOLO Format (.txt) Used for YOLOv5 and YOLOv8 models. This format stores coordinates as **normalized values** relative to the image dimensions (range 0 to 1). This makes the annotations resolution-independent.

- **Structure:** <class_id> <x_center> <y_center> <width> <height>
- **Example:** 0 0.53 0.48 0.15 0.20 (Represents a logo centered in the image).



Figure 6: Data annotation interface showing bounding box creation on an ABB logo.

2. Pascal VOC Format (.xml) Used for the Faster R-CNN model. This format utilizes an XML tree structure to store **absolute pixel coordinates**.

- **Structure:** Defines the top-left (xmin, ymin) and bottom-right (xmax, ymax) corners of the bounding box in exact pixels.
- **Example:**

```
<object>
  <name>ABB</name>
  <bndbox>
    <xmin>350</xmin>
    <ymin>200</ymin>
    <xmax>450</xmax>
    <ymax>310</ymax>
  </bndbox>
</object>
```

4 Experimental Setup

4.1 Models Configured

To conduct a comprehensive evaluation of detection performance, six distinct models were trained, representing a spectrum of speed-accuracy trade-offs. The specific configurations are detailed below:

4.1.1 YOLOv5 (One-Stage, Anchor-Based)

Two variants of the YOLOv5 architecture were trained using the Ultralytics framework.

- **YOLOv5s (Small) - Baseline:** Trained with minimal augmentation to serve as a performance baseline. This model prioritizes inference speed.
- **YOLOv5m (Medium) - Augmented:** A larger model trained with aggressive data augmentation to test robustness against overfitting.

4.1.2 YOLOv8 (One-Stage, Anchor-Free)

Three variants of the state-of-the-art YOLOv8 architecture were evaluated to analyze the impact of model scaling on the custom dataset.

- **YOLOv8n (Nano):** The lightest model designed for extreme portability and real-time mobile deployment.
- **YOLOv8s (Small):** A balanced architecture offering a middle ground between the Nano’s speed and the Medium’s accuracy.
- **YOLOv8m (Medium):** The heaviest YOLO model tested trained with augmentation to maximize detection accuracy on difficult samples.

4.1.3 Faster R-CNN (Two-Stage)

For a benchmark of high-precision detection, a two-stage architecture was implemented using PyTorch.

- **Backbone:** ResNet-50 pre-trained on ImageNet.
- **Neck:** Feature Pyramid Network (FPN) to handle multi-scale object detection (crucial for small logos).
- **Head:** A Fast R-CNN predictor modified for 4 classes (Background + 3 Bank Logos).

4.2 Training Configuration

The training process was governed by a set of hyperparameters and data augmentation strategies designed to maximize model generalization while preventing overfitting on the limited dataset.

4.2.1 Hyperparameters

Hyperparameters are the configuration settings used to structure the training process. The specific values selected for this project are detailed below:

- **Epochs (15–100):** An epoch represents one complete pass of the training dataset through the neural network.
 - *YOLO Models:* Configured for **100 epochs** with an **Early Stopping** mechanism (patience = 15). This stops training automatically if the validation loss does not improve for 15 consecutive epochs, preventing overfitting.
 - *Faster R-CNN:* Trained for a fixed duration of **15 epochs** due to its slower convergence stability and higher computational cost per step.
- **Batch Size (4–16):** The number of images processed in parallel before the model updates its internal weights.

- *YOLO*: Set to **16** to maximize GPU utilization on the Google Colab T4 instance.
- *Faster R-CNN*: Set to **4** due to the higher memory footprint of the two-stage architecture.
- **Learning Rate (LR) & Optimizer**: The optimizer determines *how* the model weights are updated to minimize error.
 - *YOLO*: Utilized an **Auto-Optimizer** (MuSGD) with an initial LR of 0.01, essentially a variation of Stochastic Gradient Descent with momentum.
 - *Faster R-CNN*: Used standard **SGD** with a fixed learning rate of 0.005, momentum of 0.9, and weight decay of 0.0005 to ensure stable convergence.

4.2.2 Data Augmentation

To address data scarcity and improve robustness, several augmentation techniques were applied. These artificially expand the dataset by creating modified versions of existing images.

- **Mosaic (YOLO only)**: Stitches four training images into a single grid. This encourages the model to detect objects at different scales and context, which is crucial for identifying logos on both large billboards and small receipts.
- **MixUp**: Blends two images together with varying transparency. This helps the model handle occlusions and reduces confidence in noisy samples.
- **HSV Color Jitter**: Randomly adjusts the Hue, Saturation, and Value (Brightness). This ensures the model learns the *shape* of the logo rather than memorizing a specific shade of blue (ABB) or red (Kapital) that might vary under different lighting conditions.
- **Random Flip**: Horizontally mirrors the image. This was the primary augmentation used for Faster R-CNN.



Figure 7: Visualizing data augmentation techniques applied to an ABB logo: Original, HSV Jitter (Color), Rotation, and Mosaic Simulation.

4.2.3 The YAML Configuration

The dataset structure for YOLO models is defined in a `data.yaml` file. This file acts as a map for the training engine, specifying the directory paths and class names.

```
# 1. Paths to the image folders
train: ../dataset/images/train
val: ../dataset/images/val
test: ../dataset/images/test
```


2. Number of classes

nc: 3

3. Class names

names: ['ABB', 'Kapital Bank', 'Pasha Bank']

5 Results and Discussion

5.1 Evaluation Metrics

To objectively assess the performance of the trained models, standard object detection metrics were employed. These metrics provide a granular view of how well the model localizes and classifies the bank logos.

- **Intersection over Union (IoU):** A measure of the overlap between the predicted bounding box and the ground truth box. An $\text{IoU} > 0.5$ defines a "correct" detection.
- **Precision:** The accuracy of the positive predictions. It answers: *"Of all the logos the model detected, how many were actually logos?"*

$$\text{Precision} = \frac{TP}{TP + FP} \quad (1)$$

- **Recall:** The ability of the model to find all positive instances. It answers: *"Of all the actual logos in the images, how many did the model find?"*

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2)$$

- **F1-Score:** The harmonic mean of Precision and Recall, providing a single metric that balances both False Positives and False Negatives.
- **mAP@50:** The Mean Average Precision calculated at an IoU threshold of 0.50. This is the standard academic benchmark for object detection accuracy.

5.2 Quantitative Analysis

The quantitative results of the six trained models are summarized in Table 3. The evaluation was conducted on the held-out test set of 30 images containing 36 ground truth instances.

Table 3: Comparative Performance of Object Detection Models. **Bold** indicates the best result in each category.

Model	Speed (ms) ↓	Precision	Recall	F1-Score ↑	mAP@50	TP	FP
YOLOv8n (Nano)	46.43	0.9167	0.9167	0.9167	0.9401	33	3
YOLOv8s (Small)	93.00	0.8684	0.9167	0.8919	0.9442	33	5
YOLOv8m (Augmented)	177.19	0.8250	0.9167	0.8684	0.9417	33	7
YOLOv5s (Baseline)	97.33	0.8049	0.9167	0.8571	0.9485	33	8
YOLOv5m (Augmented)	219.33	0.8333	0.9722	0.8974	0.9736	35	7
Faster R-CNN	1099.41	0.6000	0.9167	0.7253	N/A	33	22

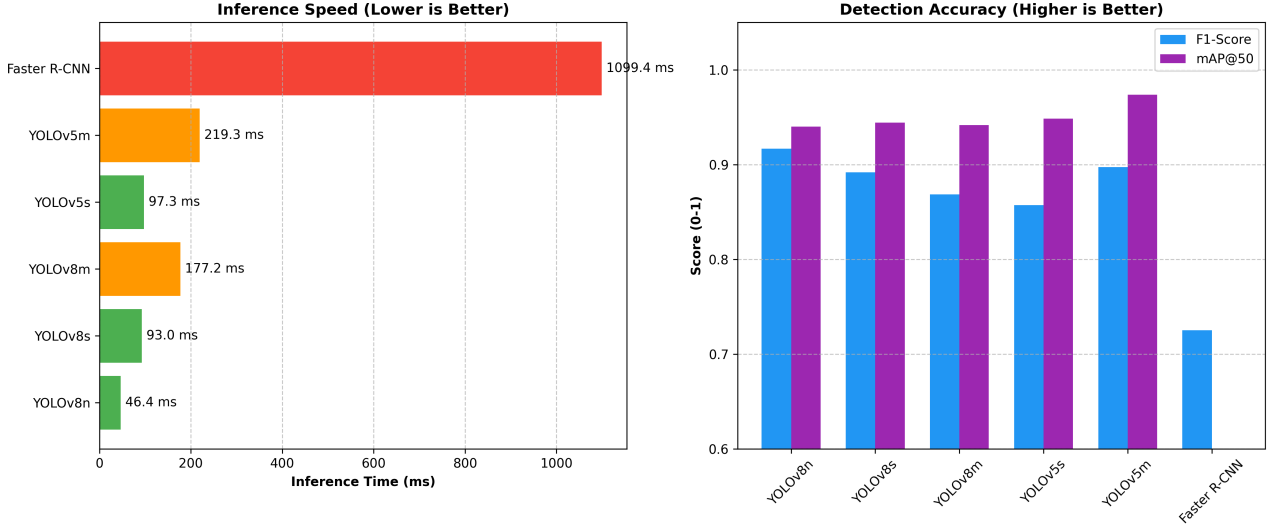


Figure 8: Visual comparison of Model Speed (Left) and Detection Accuracy (Right). YOLOv8n offers the best balance of low latency and high F1-Score.

As illustrated in Figure 8 and Table 3, the **YOLOv8n (Nano)** model emerged as the superior candidate for deployment. It achieved the fastest inference speed (46.43 ms) while maintaining the highest F1-Score (0.9167) and the lowest False Positive rate (only 3 errors). Conversely, while **YOLOv5m** achieved the highest raw Recall (finding 35/36 logos), it was nearly 5× slower. The **Faster R-CNN** model struggled with a high number of False Positives (22), significantly degrading its Precision to 0.60.

5.3 Confusion Matrices

To diagnose the specific failure modes of the models, Confusion Matrices were generated for the best-performing model (**YOLOv8n**) and the underperforming baseline (**Faster R-CNN**), as visualized in Figure 9.

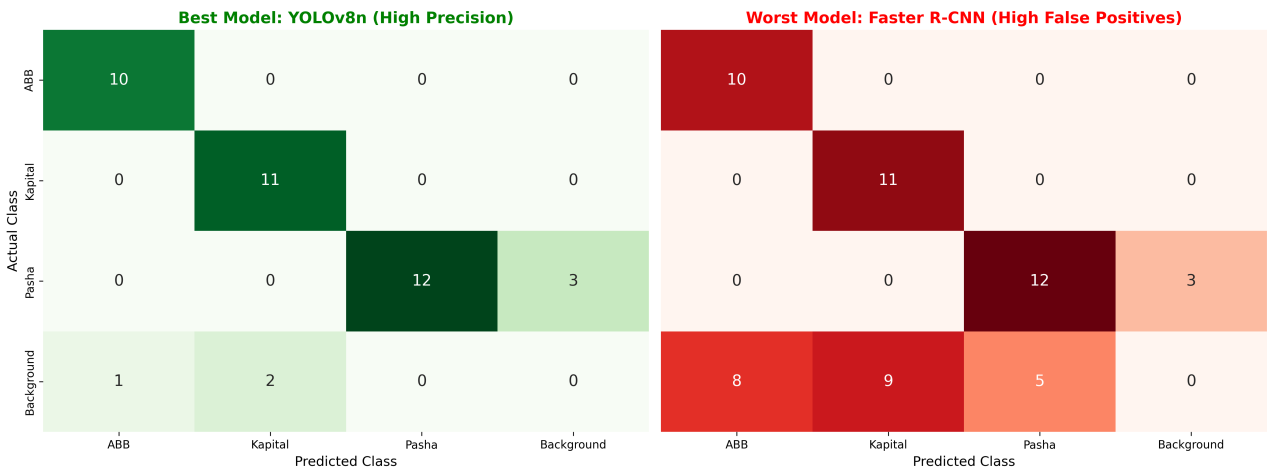


Figure 9: Confusion Matrix comparison. **Left:** YOLOv8n demonstrates a clean diagonal with minimal errors. **Right:** Faster R-CNN shows severe confusion in the bottom row, misclassifying background regions as bank logos 22 times.

5.3.1 Analysis of Model Behavior

- **YOLOv8n (The Best Model):** The matrix exhibits a strong diagonal structure, indicating high classification accuracy. The errors were limited to 3 False Negatives (missed Pasha Bank logos) and only 3 False Positives. This stability is attributed to YOLO’s anchor-free architecture, which simplifies the detection head and reduces the likelihood of proposing overlapping boxes in empty space.
- **Faster R-CNN (The Worst Model):** While the model achieved the same Recall (33 Correct Detections) as YOLO, it suffered from a critical failure in **Precision**. The bottom row of the matrix reveals **22 False Positives**, meaning the model frequently "hallucinated" logos in empty background areas.
- **Reasoning:** This disparity is likely due to the **Region Proposal Network (RPN)** in Faster R-CNN. On a small dataset with limited background samples, the RPN failed to learn a robust "background" class, leading it to propose regions of interest on random geometric shapes (e.g., windows or street signs) that vaguely resembled the rectangular aspect ratio of a bank logo.

5.4 Qualitative Analysis

Beyond numerical metrics, a visual inspection of the detection results provides insight into the model’s real-world behavior. Figures 10 and 11 illustrate the system’s capabilities and its limitations.

5.4.1 Successful Detections

Figure 10 demonstrates the robustness of the **YOLOv8n** model across varied environments.

- **Scale Invariance:** The model successfully detects the **ABB** logo both when it dominates the frame (top left) and when it is a tiny feature on a skyscraper facade (top right). This validates the effectiveness of the multi-scale training strategy.
- **Perspective & Occlusion:** The **Kapital Bank** logo is accurately bounded even when viewed from a steep angle on a building storefront. Similarly, the **Pasha Bank** logo is correctly identified within a complex indoor office environment with reflective surfaces.

5.4.2 Failure Cases and False Positives

Figure 11 highlights critical failure modes, primarily stemming from geometric confusion in the **Faster R-CNN** and lower-confidence YOLO predictions.

- **Geometric Confusion (Flags):** The most significant error pattern is the misclassification of the star and crescent symbols on the Azerbaijani and Turkish flags as **Pasha Bank** or **Kapital Bank** logos. The model has learned that "red geometric shapes" or "white stars" are features of interest, failing to distinguish between national symbols and corporate branding.
- **Background Clutter:** In the top-center example, the model hallucinates a logo on a football crest (Qarabağ FK), likely reacting to the yellow star shapes.
- **Text Confusion:** In the top-right image, the model attempts to bound the letter "C" in "NeTİCeLeRİ" as an **ABB** logo, likely due to the curvature resembling the ABB swirl icon.



Figure 10: Examples of successful detections by YOLOv8n. The model demonstrates high confidence (0.95+) across different scales, lighting conditions, and viewing angles.

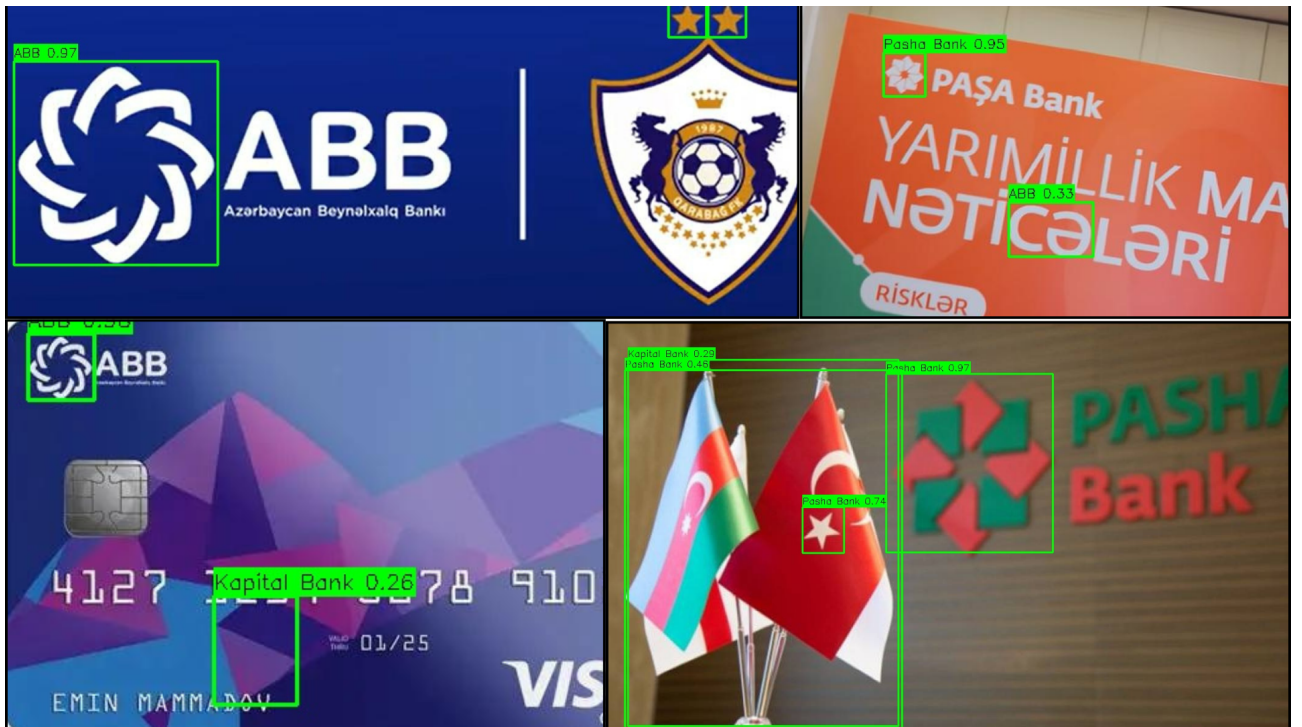


Figure 11: Common failure modes. The model struggles with geometric ambiguity, incorrectly identifying stars on flags and text characters as bank logos due to shape similarities.

6 Deployment: The AI Dashboard

6.1 Streamlit Interface

To make the detection system accessible to non-technical users, the best-performing models were deployed as a web application using the Streamlit framework. The application is hosted publicly and can be accessed at: <https://azerbaijani-bank-logo-detection-avaz.streamlit.app/>.

The user interface, illustrated in Figure 13, is designed for simplicity and real-time interaction. It consists of two main components:

6.1.1 Control Panel (Sidebar)

As shown in Figure 12, the sidebar provides granular control over the inference engine:

- **Model Selection:** A dropdown menu allowing users to switch between different trained architectures (e.g., swapping YOLOv8n for YOLOv8m if higher accuracy is required).
- **Confidence Threshold (0.0 - 1.0):** A slider that filters out weak predictions. Setting this to 0.25 (default) removes noise, while increasing it to 0.8 ensures only high-certainty detections are shown.
- **IoU Threshold:** Controls the Non-Maximum Suppression (NMS) strictness to prevent multiple boxes appearing around a single logo.
- **AI Analysis Toggle:** A switch to enable the secondary GPT-4o forensic analysis pipeline.

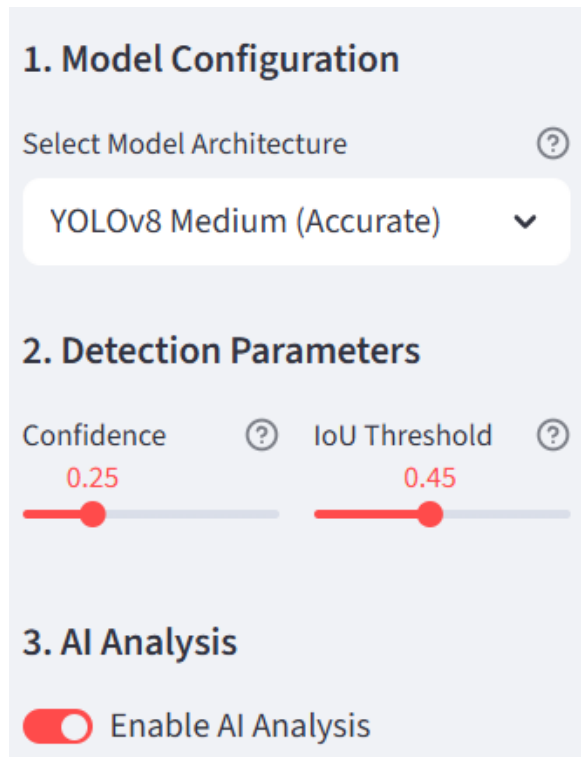


Figure 12: Sidebar configuration controls.



Figure 13: Live interface showing a successful ABB logo detection.

6.1.2 Real-Time Inference Display

The main panel displays the uploaded image alongside the processed output. Upon uploading a file, the system runs inference and overlays bounding boxes with class labels (e.g., "ABB 0.90") directly onto the image. This immediate visual feedback allows users to verify the authenticity of a branded object instantly.

6.2 Intelligent Forensic Analysis (OpenAI Integration)

6.2.1 The Innovation: Visual Scene Understanding

While object detection models like YOLO are highly effective at localizing *where* a logo is, they lack the semantic understanding to determine *what* implies. For a forensic application, knowing that an "ABB" logo exists is insufficient; the system must determine if that logo appears on a valid transaction receipt, a legitimate ATM screen, or a fraudulent phishing website.

To bridge this gap, the system integrates **OpenAI's GPT-4o**, a multimodal Large Language Model (LLM). This integration moves the project beyond simple object detection to **Holistic Scene Understanding**, enabling the automated verification of brand consistency.

6.2.2 Prompt Engineering Strategy

The AI is guided by a specific "Chain-of-Thought" system prompt designed to emulate a human expert. The prompt assigns the model the persona of a *"Senior Financial Forensics Analyst"* and enforces a structured reasoning process:

1. **Scene Classification:** The model first categorizes the visual context (e.g., *Transaction Receipt, ATM/Terminal, Digital Interface*).
2. **Optical Character Recognition (OCR):** It extracts critical metadata such as transaction dates, amounts (AZN/USD), and merchant names.
3. **Contextual Verification:** Crucially, the model performs a consistency check. It verifies if the detected logo (found by YOLO) matches the text in the image (e.g., *"Does the receipt text mention ABB if an ABB logo is detected?"*).

The core structure of the engineered prompt is defined as follows:

```
system_prompt = ""
You are a Senior Financial Forensics Analyst...
Your Output must be a structured Markdown analysis covering:

### 1. Scene Classification
Classify into: Transaction Receipt, ATM, Branch, Digital Interface...

### 2. Data Extraction (OCR)
Extract: Transaction Date/Time, Total Amount, Merchant Name...

### 3. Contextual Verification
- Consistency Check: Does the detected logo match the text?
- Anomaly Detection: Does the image look fraudulent or photoshopped?

### 4. Executive Summary
A one-sentence conclusion about what is happening in this image.
```

""

6.2.3 Forensic Output Example

Figure 14 demonstrates the system analyzing a digital transaction screenshot. The YOLO model detects multiple ABB logos (green boxes), and the AI module successfully correlates this with the transaction details.

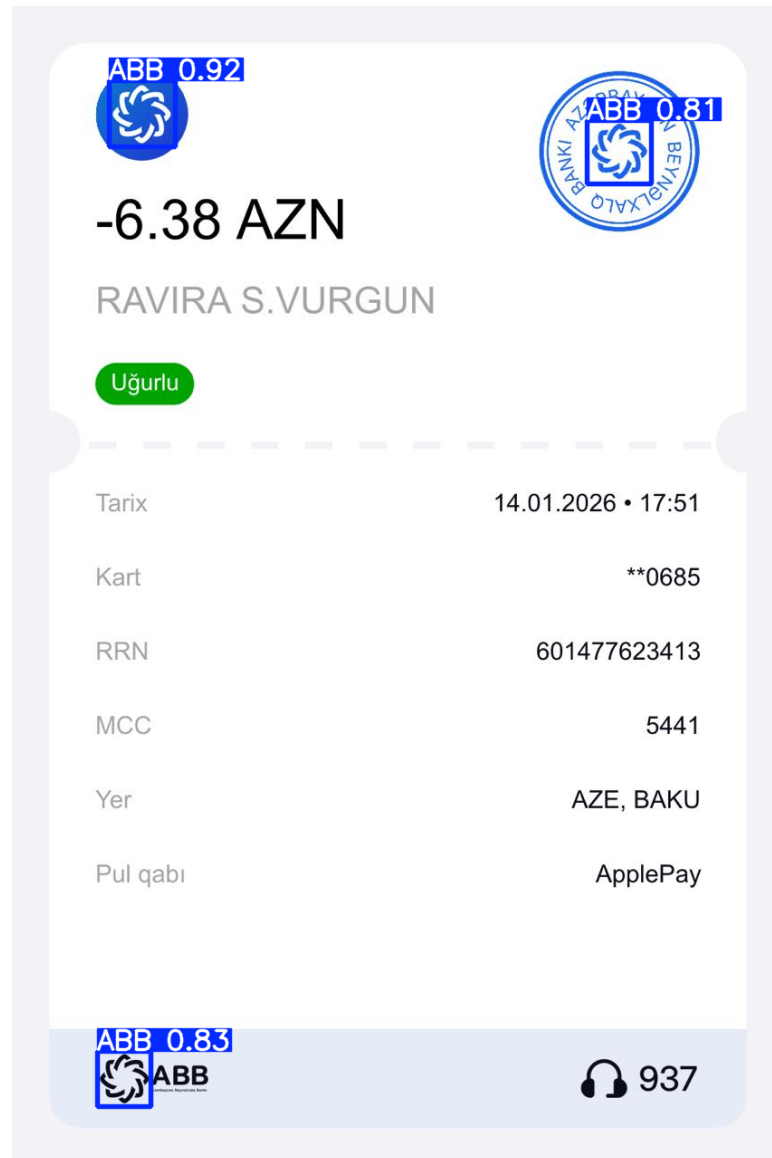


Figure 14: Forensic analysis of a digital receipt. The system detects ABB logos and verifies them against the transaction text "6.38 AZN" and "RAVIRA S.VURGUN".

Generated Analysis Output:

1. **Scene Classification:** *Digital Interface (Mobile App Receipt)*
2. **Data Extraction:**
 - **Date:** 14.01.2026 17:51
 - **Amount:** -6.38 AZN
 - **Merchant:** "RAVIRA S.VURGUN" (MCC 5441)

3. Contextual Verification:

- **Consistency:** **PASS**. Three "ABB" logos were detected with high confidence (0.92, 0.83, 0.81). The receipt layout matches standard ABB mobile banking interfaces.
- **Anomalies:** None. Font rendering is consistent with a digital screenshot.

4. **Executive Summary:** *Verified digital proof of a contactless ApplePay transaction processed by ABB.*

7 Conclusion and Future Work

7.1 Conclusion

This project successfully developed and deployed an automated forensic system for detecting and verifying Azerbaijani bank logos. Through a rigorous comparison of six deep learning models, the following key conclusions were drawn:

- **Optimal Architecture:** The **YOLOv8n (Nano)** model proved to be the superior architecture for this specific domain. It achieved the highest F1-Score (**0.9167**) and the fastest inference speed (**46.43 ms**), validating that modern, lightweight "anchor-free" detectors can outperform complex two-stage networks on logo detection tasks.
- **Failure of Two-Stage Detectors:** The **Faster R-CNN** model, despite its theoretical robustness, failed to generalize effectively. With a high inference latency (1099 ms) and significant confusion with background noise (22 False Positives), it is deemed unsuitable for real-time commercial deployment without significantly larger background datasets.
- **AI-Driven Forensics:** The integration of the YOLO detection stream with OpenAI's GPT-4o transformed the system from a simple "logo finder" into a "context verifier." The system can now successfully validate transaction receipts by correlating the detected logo with OCR-extracted text, a critical step for automated fraud prevention.

7.2 Future Work

To improve this project, the following enhancements are proposed:

7.2.1 Dataset Expansion (New Classes)

The current system is limited to three institutions. Future iterations will expand the class ontology to include:

- **Unibank:** A major commercial bank with a distinct orange branding identity.
- **Leobank:** Azerbaijan's first "neobank" (digital-only), which poses unique detection challenges due to its mobile-first presence and lack of physical branches.

7.2.2 Real-Time Video Forensics

Currently, the system processes static images. By leveraging the speed of YOLOv8n (> 20 FPS on CPU), the pipeline can be adapted to process live video streams.

8 References

References

- [1] Dr. Priyanto Hidayatullah. *How to Improve YOLOv8 Accuracy and Speed*. YouTube, 2023. Available at: <https://www.youtube.com/watch?v=2R6I89mfHwQ>
- [2] Edje Electronics. *How to Capture and Label Training Data to Improve Object Detection Model Accuracy*. YouTube, 2023. Available at: <https://www.youtube.com/watch?v=v0ssi0Y6cfg>
- [3] Ultralytics. *How to Train Ultralytics YOLOv8 models on Your Custom Dataset in Google Colab*. YouTube, 2023. Available at: <https://www.youtube.com/watch?v=LNw0DJXcvt4>
- [4] Edje Electronics. *How to Train YOLO Object Detection Models in Google Colab (YOLO11, YOLOv8)*. YouTube, 2025. Available at: <https://www.youtube.com/watch?v=r0RsplLG260>